ETAS

ETAS ISOLAR-B 12.7.0

JaCoP Scripting Guide

www.etas.com

Copyright

The data in this document may not be altered or amended without special notification from ETA S GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2024 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS ISOLAR-A/B V12.7.0 | JaCoP Scripting Guide R01 EN – 04.2025

Contents

1 About ISOLAR-B 5

2 JaCoP (Java for Configuration Processors) 6

Steps to configure Java Action in ISOLAR-B 6

2.1 Import a Project 6
2.1.1 Parameter Definitions 6

2.2 Configure EcuC Values 9
2.2.1 BSW Editor – Invocation 9
2.2.2 Create a new BSW Module (EcuC values) 9

2.3 Preparation of Java File 9

2.4 Preparation of BAMF (Build Action Manifest File) 11
2.4.1 Create a new Build Action Manifest AR Element: 12
2.4.2 Configure Build Actions using Build Action Manifest Editor 13

3 EcucValues Access 15

3.1 Access BSW/EcucValues configuration : 15

3.2 Read/Create Module: 15
3.2.1 Read Module: 15
3.2.2 Create Module: 15

3.3 Read/Create Container: 16
3.3.1 Read Container: 16
3.3.2 Create Container: 16

3.4 Read/Create Parameter: 17
3.4.1 Read Parameter: 17
3.4.2 Create Parameter: 17

3.5 Reference Parameter: 18
3.5.1 followReference() 18
3.5.2 getTargetShortName() 18
3.5.3 getTargetFullyQualifiedInstanceName() 18

3.6 Improved JaCoP apis : 19
3.6.1 Improved typesafety in createParameter() API: 19

3.7 JaCoP APIs to validate its existence : 20
3.7.1 For ECUC-Value elements 21
3.7.2 For ECUC-Definition elements 21

3.8 JaCoP API isElementReferred(final IContainingEntity containingEntity) 21

4 IArtopAbstraction Access 23

4.1 Introduction: 23

4.2 List<T> getElements(Class<T> type): 23

5 Update Model 24

5.1 Update Model 24

5.2 Pre-requisite to running Java scripts 24

5.3 Useful Java APIs 24

5.4 Invocation and Execution 26

5.5 Debugging Model Update Java scripts 28

6 Test Framework 31

6.1 Overview 31

6.2 How to write a testcase using JaCoPTestBase 31

6.3 Modifying Created Test Class 33

6.4 How to Use Util Classes 34
6.4.1 JacopTestFactory 35
6.4.2 TestGeneratorUtil 36
6.4.3 TestForwarderUtil 36
6.4.4 TestParamUtil 37
6.4.5 TestFrameworkUtil 40

6.5 Execution of testcases 42
6.5.1 How to run the testcase in GUI ? 42
6.5.2 How to run the testcase in CLI mode 43

7 Contact Information 45

1 About ISOLAR-B

ISOLAR-B is an Eclipse-based product for the configuration of ECU Basic Software. It supports the creation, editing, and updating of Basic Software configurations and the generation of Basic Software code. ISOLAR-B also supports other use cases, e.g., to develop Basic Software, and to write, debug and execute scripts. ISOLAR-B supports industry standards such as AUTOSAR as well as other formats and standards like DBC, Fibex etc.

For the development of Basic Software, this document provides information on Editors, features, and Java-based scripting of ISOLAR-B.

2 JaCoP (Java for Configuration Processors)

JaCoP is a Java-based framework of ISOLAR-B, which is used for code generation in the wider context of AUTOSAR Basic software.

JaCoP supports the following features:

1. Access BSW (Basic Software) configurations.
2. Access ASW (Application Software) configurations.
3. Forwarding - creation of new EcuC value configurations at Build Time.
4. Validation of EcuC value configurations.
5. Generation of .c, .h, and .arxml files as part of Code Generation.

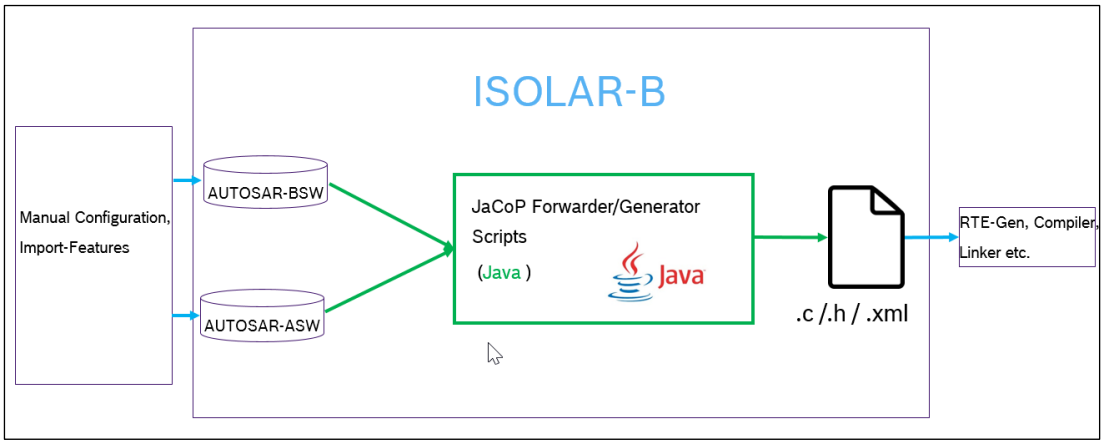

Figure 1: Overview of JaCoP as part of ISOLAR-B

Steps to configure Java Action in ISOLAR-B

List of steps:

1. Import a Project.
2. Prepare parameter definition.
3. Configure EcuC values.
4. Preparation of Java file.
5. Preparation of BAMF.
6. Execution of Java files.

2.1 Import a Project

A folder representing an AUTOSAR project can be imported into the workspace of ISOLAR-B for further configuration, as follows:

1. Open the Import wizard, **File** > **Import**.
2. Select **General** > **Import Folder as AR Project** > **Next**.
3. Select **Browse**, select the folder to be imported, name it and then click **Finish**.

2.1.1 Parameter Definitions

There are two types of parameter definitions:

1. AUTOSAR standardized parameter definitions.
2. Customer/vendor-specific parameter definitions.

2.1.1.1 AUTOSAR-standardized Parameter Definitions

AUTOSAR-standardized parameter definitions can be imported into your project by:

1. In AR Explorer **Right-click on the project > ISOLAR-A > Import ECU Param Def,** ISOLAR-A/Import EcuCParamDef wizard opens up.
2. In the wizard dialog, select **Import default Parameter Definition File.** Click **OK**

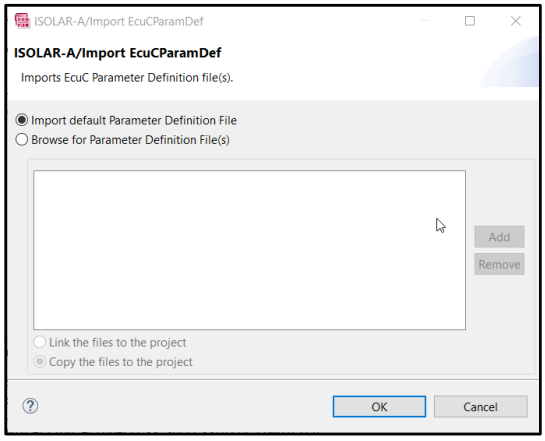

Figure 2 Import EcuCParamDef Wizard

This creates a new file *ECUConfigurationParameters.arxml* in the selected project, which contains parameter definitions of all AUTOSAR-standardized BSW modules.

2.1.1.2 Customer/Vendor-specific Parameter definitions

Customer/Vendor-specific Parameter definitions can be created in ISOLAR-B by:

1. In ECU Navigator > Expand the project > Right click on Parameter Definitions > Create Child > Create Elements | EcuC Module Def > New AR Element Creation wizard opens up.

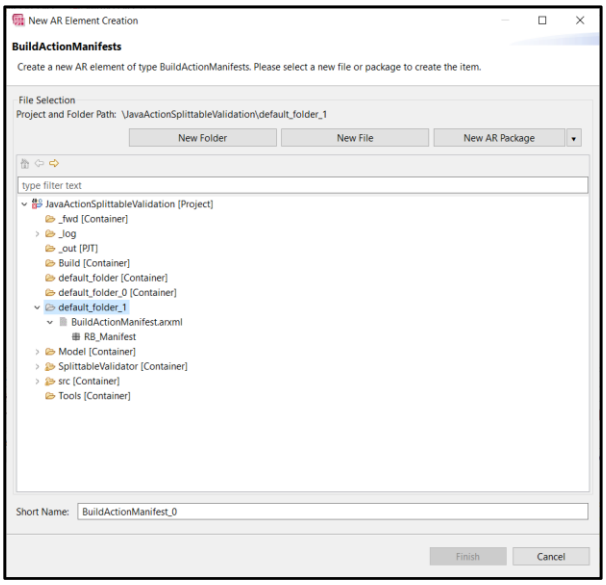

Figure 3 New AR element creation dialog

2. In New AR Element Creation Wizard, provide a name in the Short Name field, by default an auto generated module short name will come up. You can modify the short name and give short name of your choice.

3. Define the file to store the BSW module definitions , by selecting the New File Button in the wizard. This will generate a file with default name. You can rename the file by selecting the file, right click > rename. Enter the new file name and hit enter.
4. Define a new AR package for the definition by selecting the New AR Package button, keeping the selection set to the new file created. Click Finish.
5. Once this new module definition is shown in the ECU Navigator's Parameter Definitions, double-click it to open the Parameter Definition Editor. There, create a container by right clicking the module > New Child > Container > EcuC Param Conf Container Def.

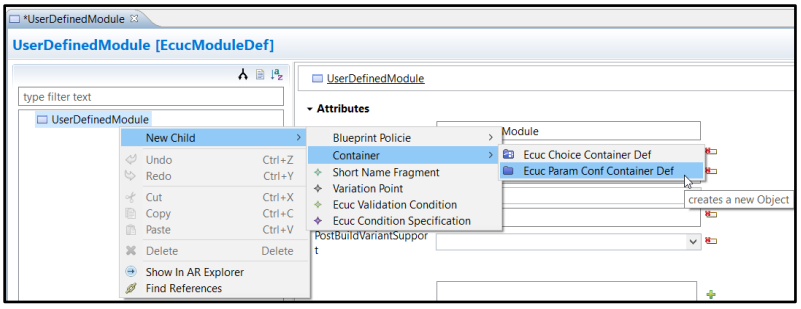

Figure 4: Creating a Container (for parameters) in the Param Definition Editor

This container can be populated with parameters as required:

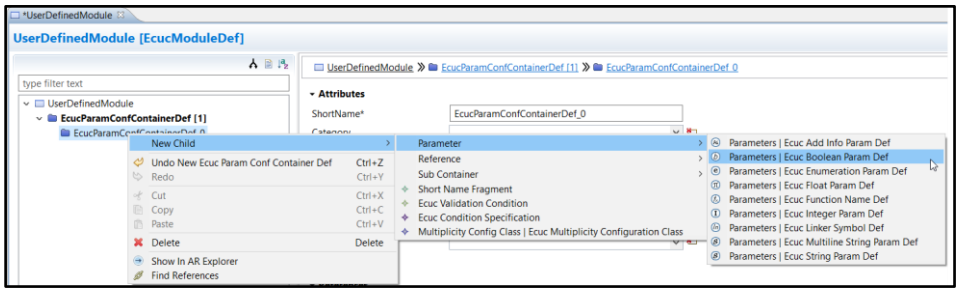

Figure 5: Adding parameters (to a container) in the Param Definition Editor

2.1.1.3 Prepare Project for Java Execution

After the creation of parameter definition, go to **ECU Navigator** > **Right click on project** > **ISOLAR-B** > **Prepare project for execute JavaActions**

This step will create the *.accessor* folder in the project which is used by Java Actions.

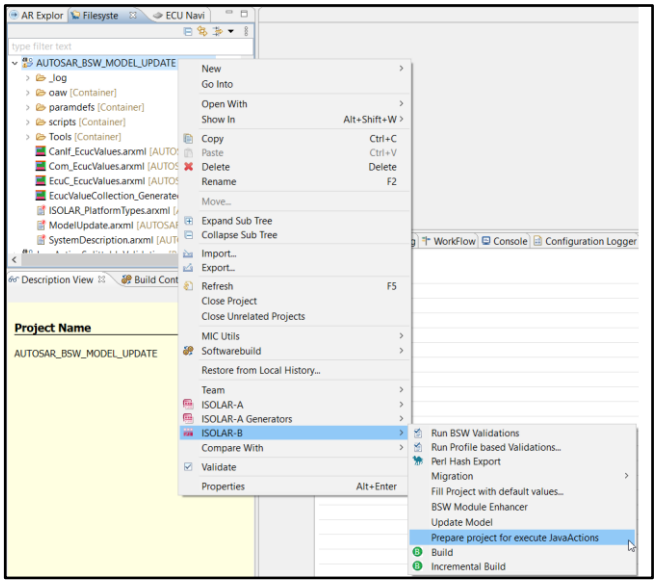

Figure 6: Prepare project for Java Execution

2.2 Configure EcuC Values

The BSW Editor of ISOLAR-B is a generic editor that supports users to configure all BSW modules and their configurable elements as defined by their parameters ('ParamDef').

It also provides a Quick Info section on the right-hand side, displaying Documentation, Post Build Variants, Annotation details for Parameters, Containers and References when selected from either the Tree view, Form or Table page of the BSW Editor.

2.2.1 BSW Editor – Invocation

The BSW Editor can be opened on BSW Elements that are displayed in the ECU Navigator:

1. Double-click any BSW Element under BSW Modules.
2. Right-click on BSW Element > Open With > BSW Editor.

2.2.2 Create a new BSW Module (EcuC values)

To create EcuC values for the BSW module that you had defined in Chapter 3, open your project in the ECU Navigator, right-click BSW Modules > Create Other Modules > Create_<UserDefinedModule>. This will open the 'New AR Element Creation' wizard.

Define a name for the new BSW module, select 'Create New AR Package' and give it a name, define a file to store the BSW module, select the folder where to store the file (inside your project), and click Finish.

To start editing this new BSW module, right-click it > Open With > BSW Editor.

2.3 Preparation of Java File

Create a folder inside your project for Java sources:

1. Create a folder for Java sources inside your project:

   File System Navigator > Right-click your project > New > Folder and name it (for e.g., *src-java*).

2. Right-click the newly created *src-java* folder **> New > Other…**

3. In the **New** wizard **>** check **Show All Wizards >** search for **class >** select **Java/Class > Next**.

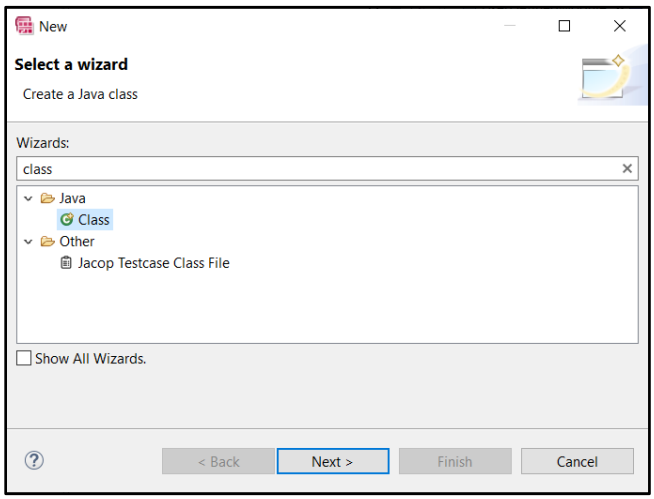

Figure 7: Select Java Class

4. **New Java Class** wizard **>** Define the Java package path (for e.g., 'bsw') **>** Define source folder **>** Define the package **>** Define the Java class name **> Add**…

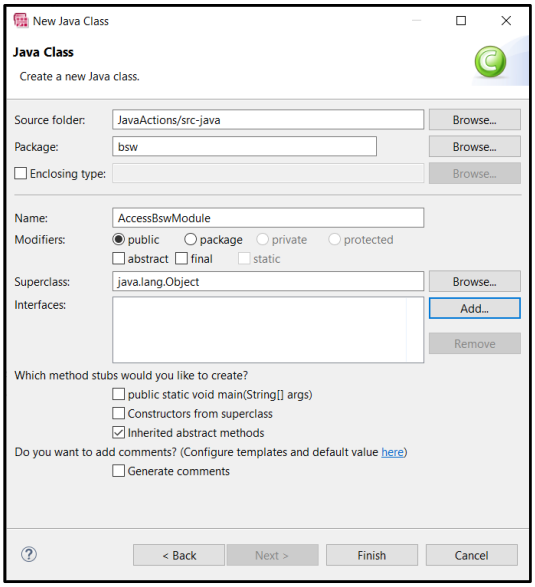

Figure 8: New Java Class wizard to help create a new Java class

5. In the **Implemented Interfaces Selection** dialog **>** search for **IJavaAction >** select the one (highlighted below in Figure 9) **>** Press **OK**.

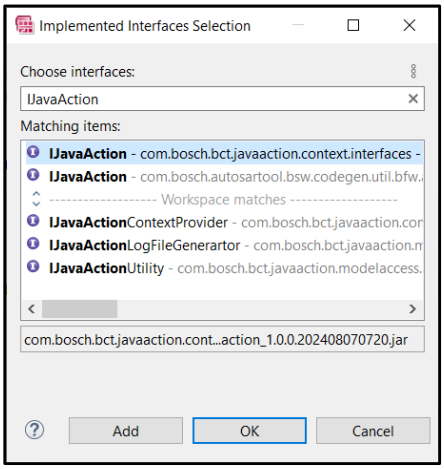

Figure 9: Select IJavaAction in **Implemented Interface Selection** dialog

As a result, a new Java class is being created. This is automatically shown in a new view:

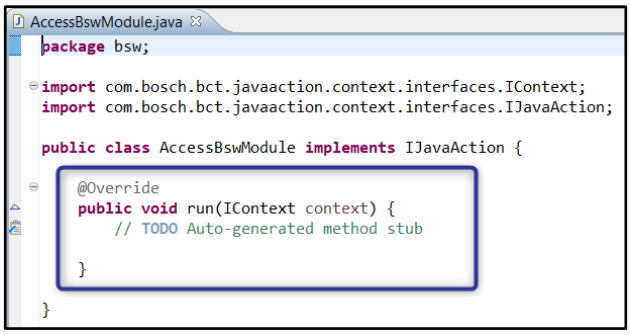

Figure 10: View showing the newly created Java class

Write the business logic to access/forward the BSW OR Generate file logic inside the highlighted run() method as shown in Figure 10.

2.4 Preparation of BAMF (Build Action Manifest File)

'Build Actions' execute Scripts, Processors, Batches, and Compilers during the "build". The sequence of executing Build Actions is (in most cases) determined by their Input and Output declarations.

For declaring all relevant Input & Output information of a Build Action that the BAMF was introduced. To properly set-up a (BFW-) build configuration, all BAMF files should contain valid (and complete) information about what data the 'Actions' are consuming and producing. Create a folder for storing BAMF File

In the File Explorer, Right-click a project **> New > Folder** (provide a name for e.g., B*AMF-Configuration*) **> Finish.** (as shown in Figure 11)

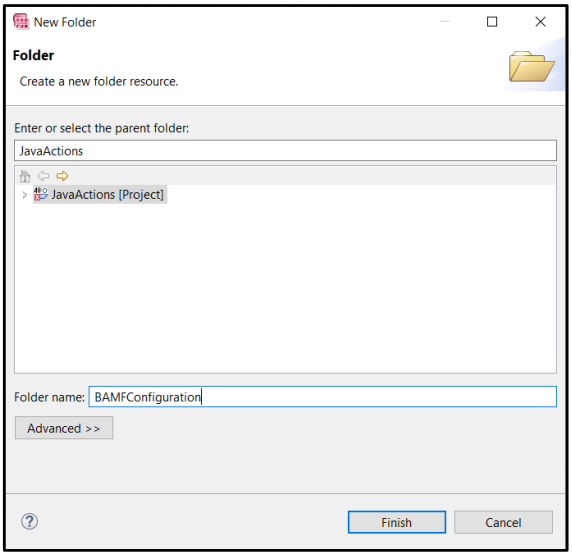

Figure 11: New Folder wizard to create a folder for a BAMF file.

2.4.1 Create a new Build Action Manifest AR Element:

1. **ECU Navigator >** Right click on **Bamf > Create BuildActionManifest > Create Elements | Build Action Manifest. New AR Element Creation** wizard opens up.

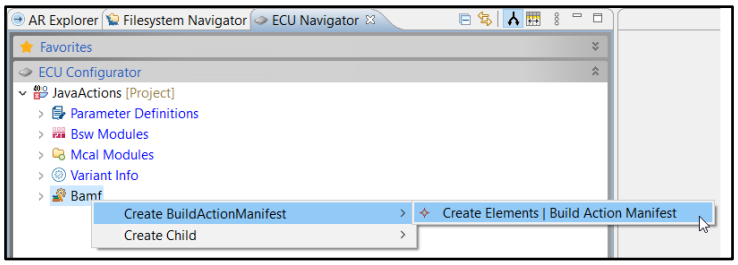

Figure 12: Launching Build Action Manifest Creation Dialog

2. In New AR Element Creation Wizard, provide a name in the Short Name field, by default an auto generated module short name will come up. You can modify the short name and give short name of your choice.
3. Define the folder under which new file for storing BSW definitions will be placed or an existing folder can be used. To create a new folder click on the the “New Folder” button in the wizard. By default, folder is generated with an default name. The folder name can be modified by selecting the folder, right **click > rename**. Enter the new folder name and hit enter.
4. Define the file to store the BSW module definitions , by selecting the New File Button in the wizard, keeping the selection on the newly created folder (or existing folder). This will generate a file with default name. You can rename the file by selecting the file, **right click > rename**. Enter the new file name and hit enter.
5. Define a new AR package for the definition by selecting the New AR Package button, keeping the selection set to the new file created. Click Finish.

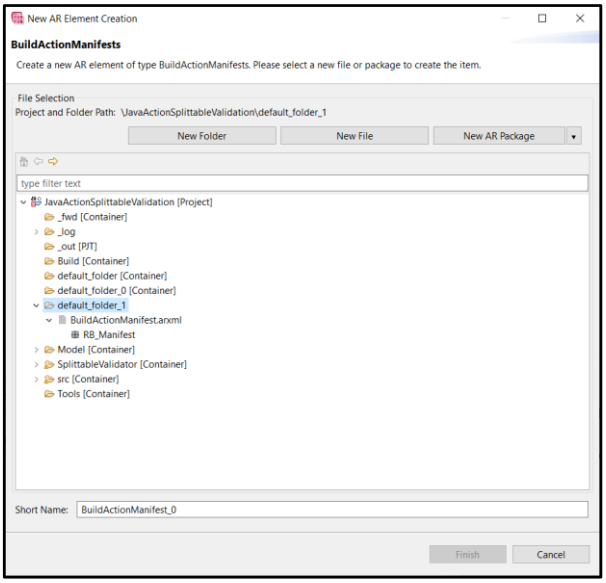

Figure 13: Build Action Manifest Creation Dialog

2.4.2 Configure Build Actions using Build Action Manifest Editor

1. **ECU Navigator > Bamf> Buildactionmanifests,** double click on the the Build action manifest, to open in Manifest Editor Page.
2. **Create new Build Action, right click on build action manifest > New Child > Build Action.**

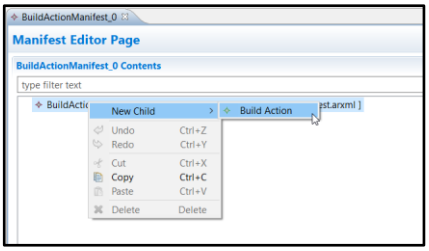

Figure 14: New Build Action creation

3. Next, to configure action parameters, **right click on build action> New Child > Input Datas | Build Action Io Element**
4. Configure the build action details section parameters.**Select the action type** (in this case: *com_bosch_bct_javaaction*)

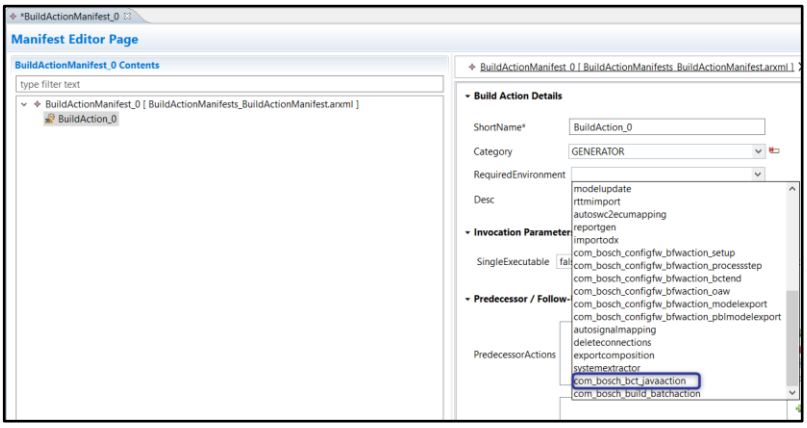

Figure 15: Specify the action type of an Action in the Manifest Editor

5. Step 4: Configure the processor for Java Action. Set input data as “ARTIFACT”, Java file name and its extension. (Java file name – Java file which implements IJavaAction):

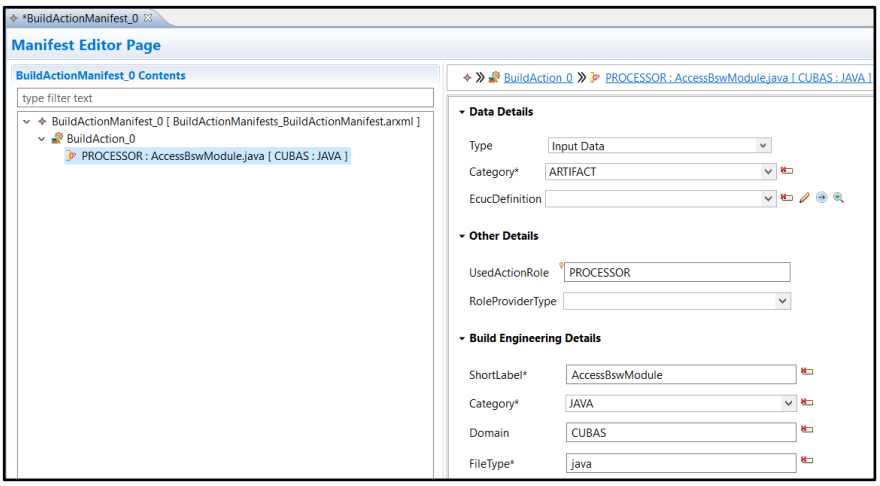

Figure 16: Define the Java Action processor in the Manifest Editor

3 EcucValues Access

3.1 Access BSW/EcucValues configuration :

The below API is used to access the BSW/EcucValues elements. This includes the forwarded elements while code generation.

EcucValues ecucValues = context.getModel(EcucValues.class);

3.2 Read/Create Module:

3.2.1 Read Module:

3.2.1.1 getModules()

This API return the all the available module configured in ECUC Value files and forwarded during code generation.

Example:

```
List<IModule> modules = ecucValues.getModules();
```

3.2.1.2 getModuleName()

This API retrn's the module configured in ECUC Value files and forwarded during code generation.

ModuleName - name of the module to be accessed.

Example:

```
EcucValues ecucValues = context.getModel(EcucValues.class);
Can canModule = ecucValues.getCan();
```

3.2.1.3 getModulesByDefinitionName(String moduleDefinitionName)

This API retrn's the module configured in ECUC Value files and forwarded during code generation with the definition shortname - moduleDefinitionName

moduleDefinitionName - definition name of the module to be accessed.

Example:

```
EcucValues ecucValues = context.getModel(EcucValues.class);
List<IModule> modulesByDefinitionName =
values.getModulesByDefinitionName("Can");
```

3.2.2 Create Module:

3.2.2.1 createModuleName();

This API creates/forward the module with default definition name as short name.

ecucValues.createModuleName(String Shortname);

This API creates/forward the module with shortname mentioned as an input parameter.

3.2.2.2 createModuleName(String Shortname,VariationPoint variationPoint)

This API creates/forward the module with shortname and variationPoint as an input parameters.

ecucValues.createModuleName(String Shortname, VariationPoint variationPoint);

Example:

```
VariationPoint variationPoint =
VarianthandlingFactory.eINSTANCE.createVariationPoint();
variationPoint.setShortLabel("Label");

ConditionByFormula conditionByFormula =
VarianthandlingFactory.eINSTANCE.createConditionByFormula();
conditionByFormula.setMixedText("1==1");

variationPoint.setSwSyscond(conditionByFormula);
ecucValues.createCan("SampleCan", variationPoint);
```

3.3 Read/Create Container:

3.3.1 Read Container:

3.3.1.1 getContainers()

This API returns the all the available module configured in ECUC Value files and forwarded during code generation.

Example:

```
Can canModule= ecucValues.geCan();
List<IContainer> containerList =  canModule.getContainers()
```

3.3.1.2 getContainerName()

This API return's the module configured in ECUC Value files and forwarded during code generation.

ContainerName - name of the module to be accessed.

Example:

```
Can canModule= ecucValues.geCan();
CanConfigSet canConfigSetObject =  canModule.getCanConfigSet()
```

3.3.1.3 getContainersByDefinitionName(String containerDefinitionName)

This API return's the Container configured in ECUC Value files and forwarded during code generation with the definition shortname - containerDefinitionName

containerDefinitionName - definition name of the module to be accessed.

Example:

```
EcucValues ecucValues = context.getModel(EcucValues.class);
Can can = values.getCan();
List<IContainer> containersByDefinitionName =
can.getContainersByDefinitionName("CanConfigSet");
```

3.3.2 Create Container:

3.3.2.1 module.createModuleName();

This API creates/forward the module with default definition name as short name.

ecucValues.createModuleName(String Shortname);

This API creates/forward the module with shortname mentioned as an input parameter.

3.3.2.2 module.createContainerName(String Shortname,VariationPoint variation-Point)

This API creates/forward the container with shortname and variationPoint as an input parameters.

module.createContainerName(String Shortname, VariationPoint variationPoint);

Example:

```
VariationPoint variationPoint = VarianthandlingFacto-
ry.eINSTANCE.createVariationPoint();
variationPoint.setShortLabel("Label");

ConditionByFormula conditionByFormula = VarianthandlingFacto-
ry.eINSTANCE.createConditionByFormula();
conditionByFormula.setMixedText("2==2");

variationPoint.setSwSyscond(conditionByFormula);
canmodule.createCanConfigSet("SampleConfigSet", variationPoint);
```

3.4 Read/Create Parameter:

3.4.1 Read Parameter:

3.4.1.1 container.getParameters():

This API return the all the available parameters configured in container.

Example:

```
Can canModule= ecucValues.geCan();
CanConfigSet canConfigSetObject = canModule.getCanConfigSet();
List<IParameter> parameters = canConfigSetObject.getParameters();
```

NOTE: In code generation context, if any element has forwarded value available this API shall return the forwarded value, whereas in case of "WithAuto" parameters if it has a configured value with "isAutoValue" attribute not configured or set to FALSE, this API shall return the configured value even if the forwarded value exists.

3.4.1.2 container.getParameterName():

This API return’s the Parameter configured in ECUC Value files and forwarded during code generation.

ContainerName - name of the module to be accessed.

Example:

```
IIntegerParameter canControllerId = canController.getCanControllerId();
```

NOTE: In code generation context, if any element has forwarded value available this API shall return the forwarded value, whereas in case of "WithAuto" parameters if it has a configured value with "isAutoValue" attribute not configured or set to FALSE, this API shall return the configured value even if the forwarded value exists.

3.4.2 Create Parameter:

3.4.2.1 container.createParameterName(String parametervalue);

This API creates/forward the module with default definition name as short name.

ecucValues.createModuleName(String Shortname);

This API creates/forward the module with shortname mentioned as an input parameter.

3.4.2.2 container.createParameterName(String parametervalue,VariationPoint);

This API creates/forward the parameter with parametervalue and variationPoint as an input parameters.

container.createParameter(String parametervalue, VariationPoint variationPoint);

Example:

```
VariationPoint variationPoint = VarianthandlingFacto-
ry.eINSTANCE.createVariationPoint();
variationPoint.setShortLabel("Label");

ConditionByFormula conditionByFormula = VarianthandlingFacto-
ry.eINSTANCE.createConditionByFormula();
conditionByFormula.setMixedText("1==1");

variationPoint.setSwSyscond(conditionByFormula);
canhardwareobject.createCanBufferId(100, Variation Point);
```

3.5 Reference Parameter:

Reference parameter has below mentioned API's.

3.5.1 followReference()

This API return the Target Container/Module configured as a value in the Reference Parameter.

Caution: This API have performance issue for Foreign Reference Parameter's. To get the shotname/FullyQualifiedName use the getTargetShortName() or getTargetFullyQualifiedInstanceName() API based on the requirment.

syntex:

IReferenceParameter.followReference()

Example:

```
IReferenceParameter canControllerRef =
can.getCanConfigSet().getCanHardwareObjects().get(0).getCanControllerRef();
IBswElement targetElementObject = canControllerRef.followReference();
```

3.5.2 getTargetShortName()

Return the shortname of the Reference Parameter value(Target Element)

syntex:

IReferenceParameter.getTargetShortName()

Example:

```
IReferenceParameter canControllerRef =
can.getCanConfigSet().getCanHardwareObjects().get(0).getCanControllerRef();
String shortName = canControllerRef.getTargetShortName();
```

3.5.3 getTargetFullyQualifiedInstanceName()

Return the FullyQualifiedName of the Reference Parameter value(Target Element)

syntex:

IReferenceParameter.getTargetFullyQualifiedInstanceName()

Example:

```
IReferenceParameter canControllerRef =
can.getCanConfigSet().getCanHardwareObjects().get(0).getCanControllerRef();
String fullyQualifiedInstanceName = canController-
Ref.getTargetFullyQualifiedInstanceName();
```

3.6 Improved JaCoP apis :

Typesafety improvements in JaCoP accessors allows,
- Script owners to provide appropriate value for creating parameters
- Return specific type referenceValue thereby avoiding unwanted casting operation in the script.

3.6.1 Improved typesafety in createParameter() API:

3.6.1.1 For Parameter

Before Change:

```
Parameter.createParameter(Object value);
```

After Change:

1. For Integer Parameter:

```
Parameter.createParameter(Long value);
Parameter.createParameter(BigInteger value);
Parameter.createParameter(Object value);
```

2. For Boolean Parameter:

```
Parameter.createParameter(Boolean value);
Parameter.createParameter(Object value);
```

3. For Float Parameter:

```
Parameter.createParameter(Double value);
Parameter.createParameter(Object value);
```

4. For String Parameter:

```
Parameter.createParameter(String value);
Parameter.createParameter(Object value);
```

5. For Internal Reference Parameter:
Parameter whose definition is of type EcuCReference, Choice Reference, Symbolic Reference, Uri Reference.

3.6.1.2 For Reference Parameter

Before Change:

```
RefParameter.createParameter(String value);
RefParameter.createParameter(Object value);
```

After Change :

```
RefParameter.createParameter(IContainer value);
RefParameter.createParameter(IModule value);
RefParameter.createParameter(String value);
RefParameter.createParameter(Object value);
```

3.6.1.3 For Foreign RefParameter:

Before Change:

```
RefParameter.createParameter(String value);
RefParameter.createParameter(Object value);
```

After Change:

```
RefParameter.createParameter(Referrable value);
RefParameter.createParameter(String value);
RefParameter.createParameter(Object value);
```

3.6.1.4 Improve typesafety in getReferrerContainers() API:

Before Change :

```
<T> List<T> IContainer.getReferrerContainers(Class<T> classOfContainer)
```

After Change:

Improve typesafety in followReference () API:

```
<T extends IContainer> List<T> IContainer.getReferrerContainers(Class<T>
classOfContainer)
```

3.6.1.5 For Internal ReferenceParameter:

Parameter whose definition is of type EcuCReference, Choice Reference, Symbolic Reference,Uri Reference.

Before Change:

```
IBswElement followReference()
```

After Change:

```
IBswElement followReference()
T followReference(Class<T> classOfTarget)
```

3.6.1.6 For Foreign RefParameter:

Before Change:

```
Object followReference()
```

After Change:

```
Object followReference()
T followReference(Class<T> classOfTarget)
```

3.6.1.7 For Instance Foreign RefParameter:

Before Change:

```
AtpFeature followReference()
```

After Change:

```
AtpFeature followReference()
T followReference(Class<T> classOfTarget)
```

3.7 JaCoP APIs to validate its existence :

The below API is used to check if BSW/EcucValues elements exists. This includes the forwarded BSW/EcucValues elements during code generation. The API can used to check for all module, container, parameter and reference ecucValues elements.

3.7.1 For ECUC-Value elements

The below APIs are used to check if BSW ECUC-Value elements exists or not. This includes the forwarded BSW ECUC-Value elements during code generation ~~as well~~. This API can be used to check for any module, container, parameter and reference elements.

3.7.1.1 isElementExist(ecucValueShortName)

To check if an element exists with its short name passed as an argument.

For example,

```
JaCopModelUtil jacopModelUtil = new JaCopModelUtil();
jacopModelUtil.isElementExist("Fee");
```

3.7.1.2 isElementExist(ecucValueShortName, ecucValueDefinitionFqn);

To check if an element exists with its short name and definition fully qualified name passed as an argument.

For example,

```
JaCopModelUtil jacopModelUtil = new JaCopModelUtil();
jacopModelUtil.isElementExist("Fee", /AUTOSAR/EcucDefs/Fee");
```

3.7.2 For ECUC-Definition elements

The below APIs are used to check if BSW parameter definition elements exists or not. This API can be used to check for any module, container, parameter and reference definition elements.

3.7.2.1 isDefElementExist(defnShortName);

To check if an element exists with its definition short name passed as an argument.

For example,

```
JaCopModelUtil jacopModelUtil = new JaCopModelUtil();
jacopModelUtil.isDefElementExist("FeeGeneral");
```

3.7.2.2 isDefElementExist(defnShortName, ecucValueDefinitionFqn);

To check if an element exists with its definition short name and definition fully qualified name passed as an argument.

For example,

```
JaCopModelUtil jacopModelUtil = new JaCopModelUtil();
jacopModelUtil.isDefElementExist("FeeGeneral",
"/AUTOSAR_Fee/EcucModuleDefs/Fee/FeeGeneral");
```

3.8 JaCoP API isElementReferred(final IContainingEntity containingEntity)

The isElementReferred API is introduced to check if any Module or Container element is being referred by any other element(s) in the project. This API is supported in code generation, Importer and update model features.

NOTE: In code generation context, this API does not rely on the INPUT / OUTPUT data defined in the BAMF file and shall fetch the data if the element is referred or not across the project.

Example:

isElemlentReferredExample

```
JaCopModelUtil jacopModelUtil = new JaCopModelUtil();
EcuC ecuc = ecucValues.createEcuC("EcuC_1");
FeeGeneral feeGeneral = ecucValues.getFee().getFeeGeneral();
assertTrue(jacopModelUtil.isElementReferred(feeGeneral));
assertFalse(jacopModelUtil.isElementReferred(ecuc));
```

isElementReferred API accepts IContainingEntity which is either module or container element and returns Boolean true if the argument passed is referred in the project or returns Boolean false otherwise.

4 IArtopAbstraction Access

4.1 Introduction:

ASW/ARTOP elements can be accessed through IArtopAbstraction Interface. There are API's available in this interface to access / create ARTOP Elements. Below is an example script for accessing ASW/ARTOP elements.

```
// Create ARTOP Abstraction Object
IArtopAbstraction artop = context.getModel(IArtopAbstraction.class);
// Access ARTOP Elements through getElements API
List<CompositionSwComponentType> compositions =
artop.getElements(CompositionSwComponentType.class);
List<SwComponentPrototype> components =
artop.getElements(SwComponentPrototype.class);
```

4.2 List<T> getElements(Class<T> type):

This API returns the list of non-merged elements of class type "type" by default and is controlled with a switch "**com.bosch.bsw.merge_artop_elements**", which when set to true in the properties file (buildSettings.properties) returns the merged elements.

5 Update Model

5.1 Update Model

This feature allows you to execute multiple oAW/Java scripts to perform a Model Update.

5.2 Pre-requisite to running Java scripts

When using Java scripts to perform the Model Update, the following pre-requisites need to be satisfied:

1. Create an empty marker file with a *.javapath* extension and save it in the Java script path

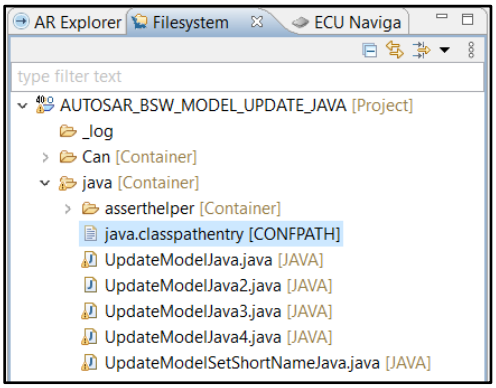

Figure 17 Creating an empty marker file in the Java script path

2.Right-click on the Project and select ISOLAR-B > Prepare project for execute Javaactions:

5.3 Useful Java APIs

APIs in IModelUpdateAPIs class

getFileParserAPIs(): IFileParser	Return: File parser to work with file contents

APIs in IFileParser class

readAllLines(String): List<String>	Specifies an absolute path to the file, and returns all lines in the specified file as a list.
getLine(String,int): String	Specifies an absolute path to the file and a line number. Returns the specified line
getLines(String,int,int): List<String>	Specifies an absolute path to the file, and a beginning and ending line number. Returns the specified line
getPatternMatchingLines(String,String): List<String>	Specifies an absolute path to the file and a search pa?ern. Returns all lines in the file that match the search pattern
convertCSVToModel(String,boolean): ICsvMode	Specifies an absolute path to the file and whether the first row is a header with column names. Returns converted CSV

	model from the file content assuming a comma as the field separator.
convertCSVToModel(String,Boolean,char): ICsvModel	Specifies an absolute path to the file, whether the first row is a header with column names, and the field delimiter. Returns converted CSV model from the file content using the specified field delimiter as the separator.
convertExcelToCSVModel(String,String,Boolean): ICsvMode	Specifies an absolute path to an xlsx file, the name of the Excel sheet, and whether the first row is a header with column names. Returns converted CSV model from the contents of the xlsx file.

APIs in ICsvModel class

getNumberOfRows(): int	Returns the number of rows from the parsed CSV file
getNumberOfColumns(): int	Returns the number of columns from the parsed CSV file
getRow(int): String[]	Returns an array of the values in the specified row number
getColumn(int): String[]	Returns an array of the values in the specified column number.
getColumn(String): String[]	Returns an array of the values in the specified column name.
getValue(int,int): String	Returns the value from the specified row and column name.
getValue(int,String): String	Returns the value from the specified row and column number
getColumnNames(): String[]	Returns an array with the column names
getColumnNumber(String): int	Returns the column number of the column matching the specified name.

JaCoP Supported APIs

API used to access BSW/EcucValues elements	EcucValues ecucValues = context.getModel(EcucValues.class)
Fetch Modules from ECUC Model	- List<IModule> modules = ecucValues.getModules() - List<IModule> modules = ecucValues.getModulesByDefinitionName(definition name) - Can canModule = ecucValues.getCan()
Create Modules	- Can canModule = values.createCan(shortname) - Can canModule = values.createCan()
Remove Modules	- remove()

Fetch Containers	- List<IContainer> containers = can.getContainers() - List<IContainer> containers = can.getContainersByDefinition-Name(definitionName) - CanConfigSet canConfig = can.getCanConfigSet()
Create Containers	- CanConfigSet canConfig = can.createCanConfigSet()
Remove Containers	- remove()
Fetch Parameters	- List<IParameter> containers = can.getParameters() - List<IParameter> containers = can.getParametersByDefinition-Name(definitionName) - getCanControllerId()
Create Parameters	- CanConfigSet canConfig = canController.createCanBusOffProc-cessing(value)
Modify Parameter Value	- overwriteParameter(Parameter Name, Value)
Fetch Parameter Value	- o getCanControllerId().getValue()
Remove Parameters	- getCanControllerId().remove()
Modify Short Names	- setShortName(name) -> Modifies the Short Name of container or module - Scope : Module and Container
Fetch Origins	- getOriginis() -> Returns the list of value file names where the element is configured - Scope : Module, Container and Parameter
Get Parent Element	- getParent() -> Returns the parent element of container or module - Scope : Module and container
Fetch Definition	- getDefinition() -> Returns the definition element - Scope : Module, Container and Parameter
Available APIs on Defini-tion element	- getLowerMultiplicity() -> Returns the lower multiplicity - getUpperMultiplicity() -> Returns the upper multiplicity - isModule() -> Returns true for Modules - isContainer() -> Returns true for Container - isParameter() -> Returns true for Parameter - isUpperMultiplicityInfinite() -> Returns true for infiniteupper multiplicity
System Condition Operation	- createSyscond(String Formula) -> Create a system condition with the given formula - getSysCond() -> Returns the system condition formula - Scope : Module, Container and Parameter

5.4 Invocation and Execution

Right-click on the **Project** and select **ISOLAR-B** > **Update Model** to open the **Update Model** dialog window

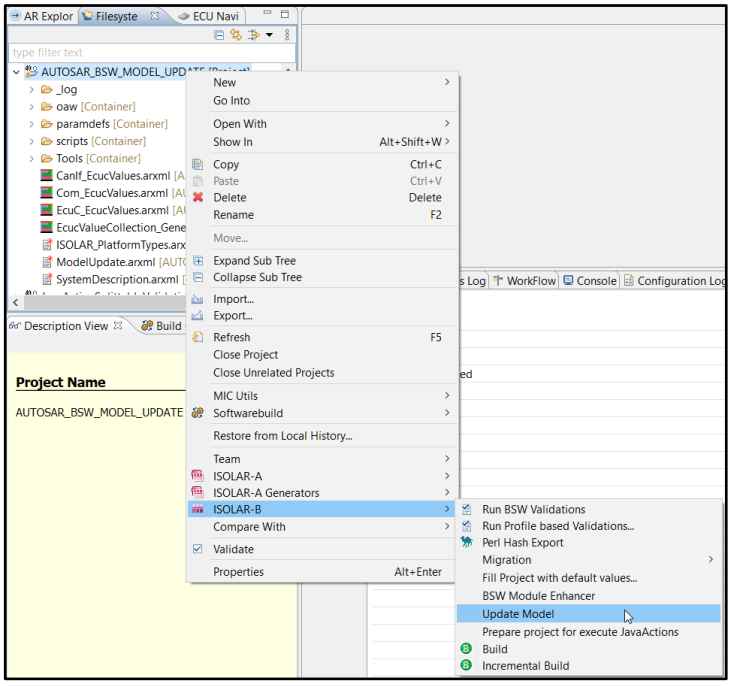

Figure 18 Invoking Update Model

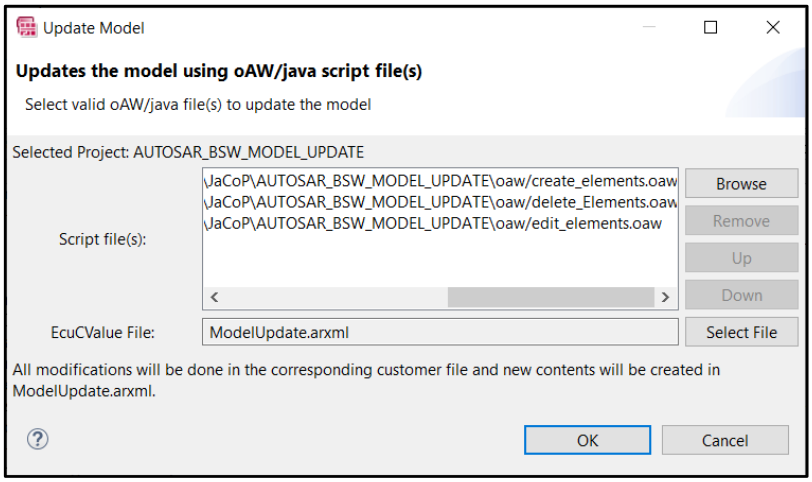

Figure 19 Model Update dialog

As shown above, the .oAW script file(s) and .java file(s) to be executed by the Model Up date are selected via the Browse button. The order in which the scripts are executed can be altered using the Up/Down buttons.

Click OK and a Progress bar will display the execution status:

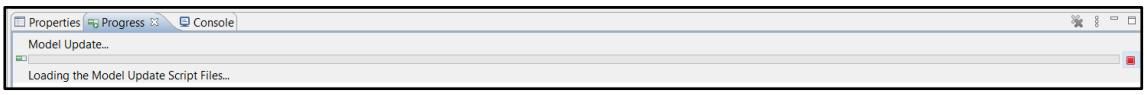

Figure 20 Progress of Model Update

The following dialog window appears once the script has completed succesfully (with the changes available in the ECUC-Value file.).

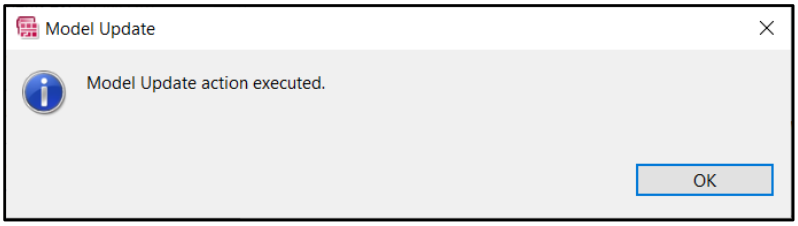

Figure 21 Model Update execution completed

5.5 Debugging Model Update Java scripts

Use the following steps to debug your Model Update java scripts.

1.Set a breakpoint in the Java file.

2.Search for, and then open “Debug Configurations…”

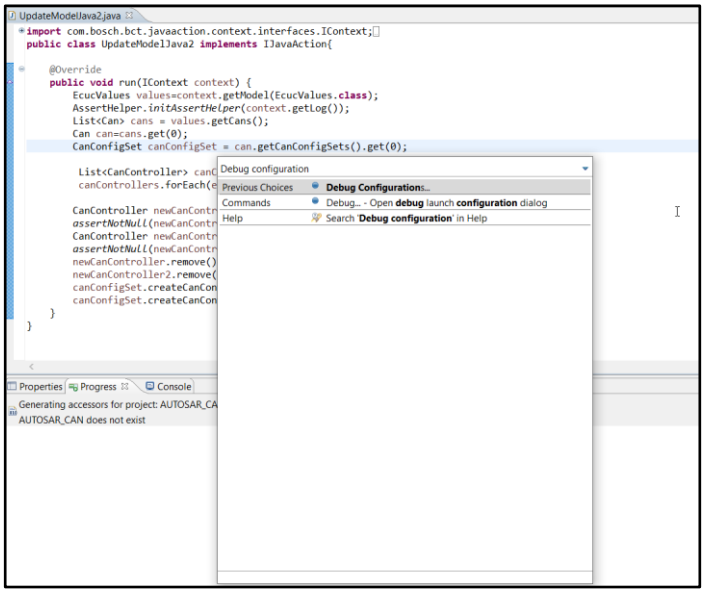

Figure 22 Opening Debug Configuration

3.In the Debug Configurations dialog, create a new Eclipse Application Launch Configuration.

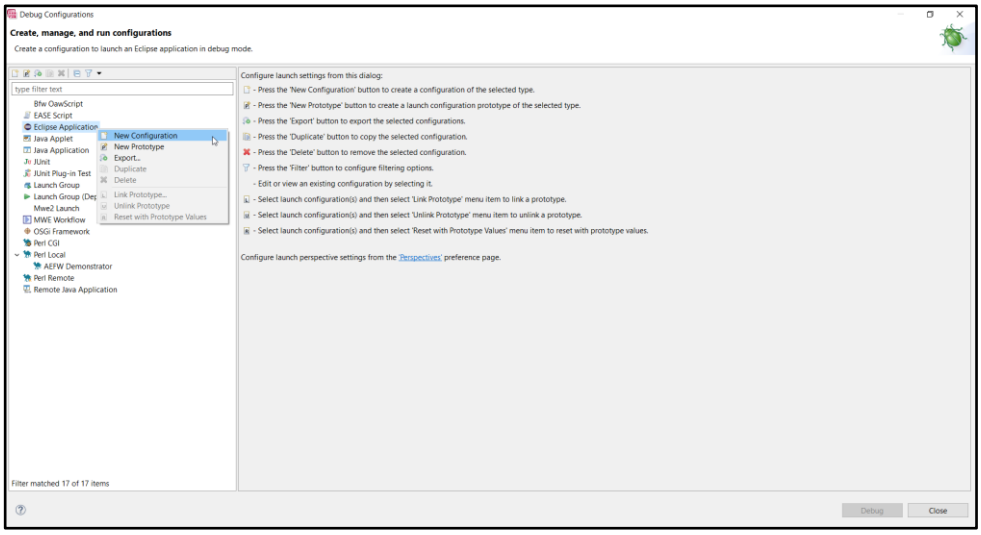

Figure 23 Opening debug configuration

4.In the Main tab, enter a name for the new Launch Configuration, select the "Run a product" option and enter "com.bosch.autosartool.bsw.ui.perspective. product" as the product name.

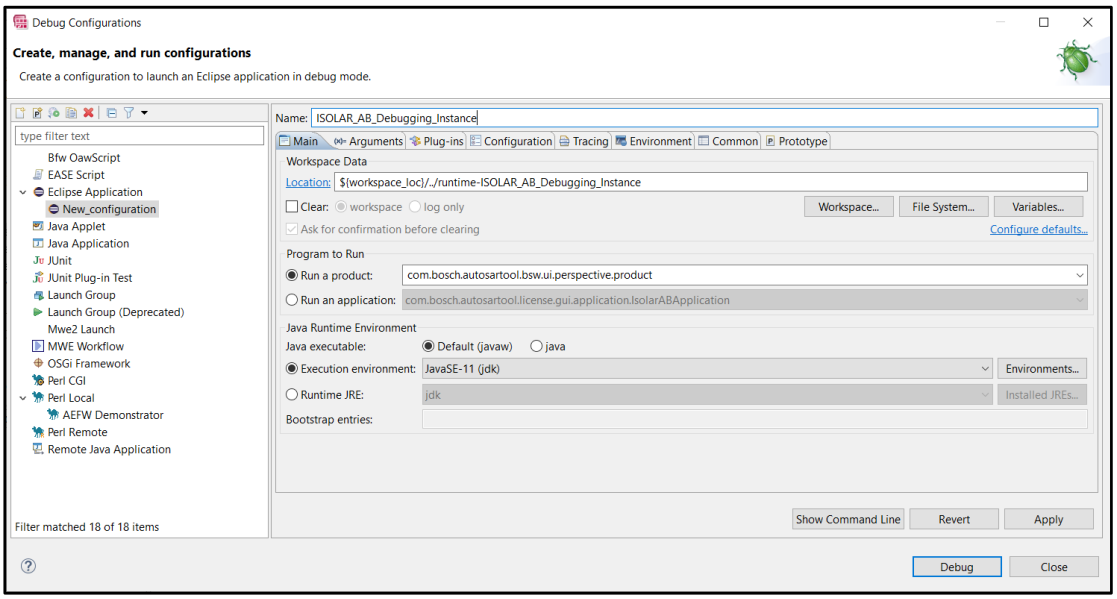

Figure 24 Debug configuration settings

5.In the Plug-ins tab, unselect the "Validate Plug-ins automatically prior to launching" option.

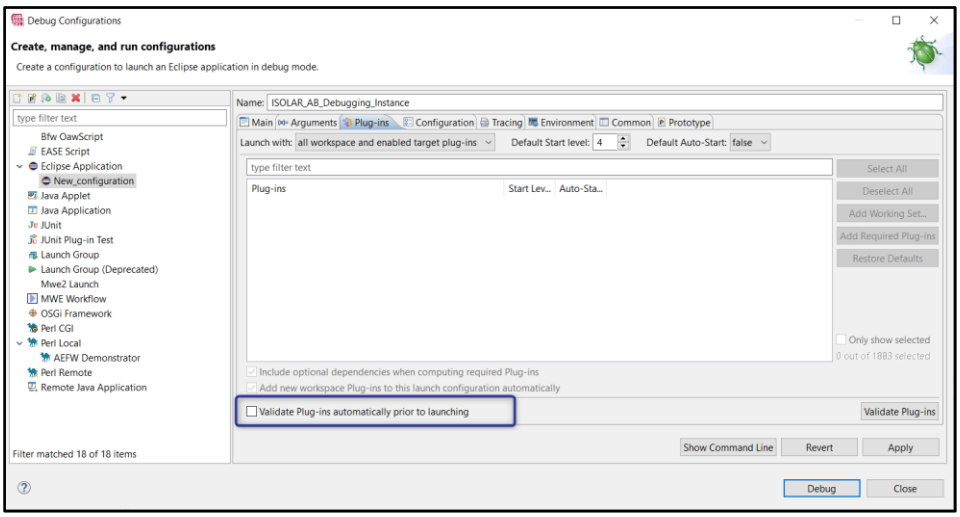

Figure 25 Deselecting plugin validation

6.Click "Debug" to launch a new ISOLAR-AB instance.

7.In the new instance, create a project with the same location as that of original instance.

8.Perform the Model Update in the new instance.

9.If the breakpoint that you set in the original instance is encountered, execution will be halted and you will be prompted to switch to the Debug perspective.

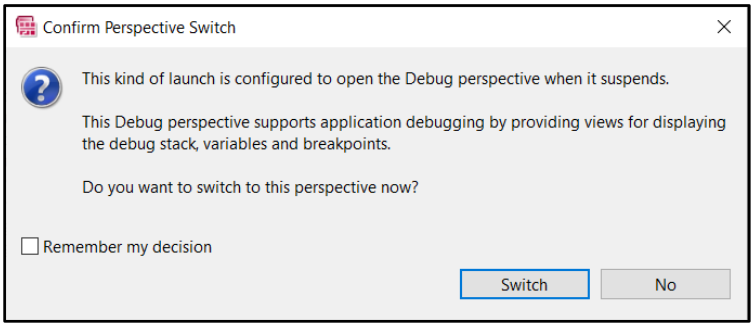

Figure 26 Confirmation dialog to switch perspective.

10.To Display the source code, click Add… in the “Edit Source Lookup Path” dialog, select Java Project in the “Add Source” dialog, choose your project, then click OK.

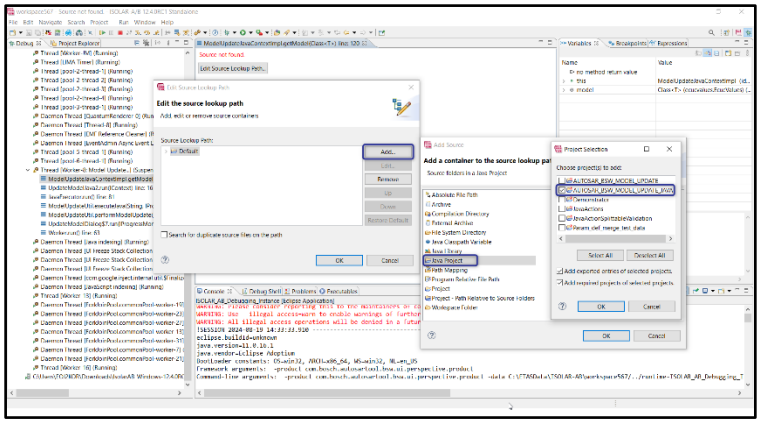

Figure 27 Java Source project selection

6 Test Framework

6.1 Overview

JaCoP Test framework allows both BSW/ASW developers to test the java script execution in a code generation context. Here, a junit testbase for the developers is introduced, on top of which the developer can write the test method to check the execution.

6.2 How to write a testcase using JaCoPTestBase

Step-1: Import the Project into the workspace using **File > Import > General / Exixting Projects Into Workspace.** Click Next.

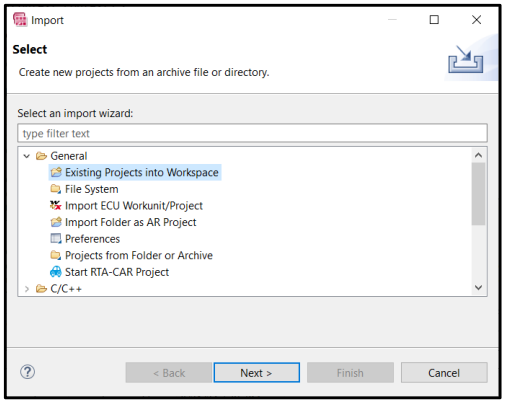

Figure 28 Import project wizard

In Import wizard click on Browse button, select the project, and click on Finish.

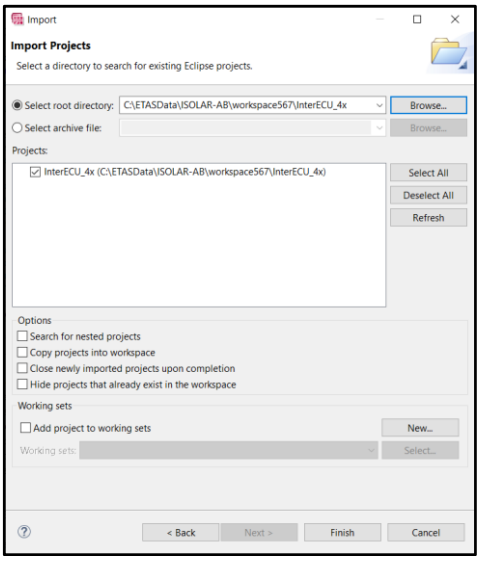

Figure 29 Import Wizard to browse the project to be imported

Step-2: Create a new JaCoP testcase class file, using File > New > Jacop Testcase Class File.

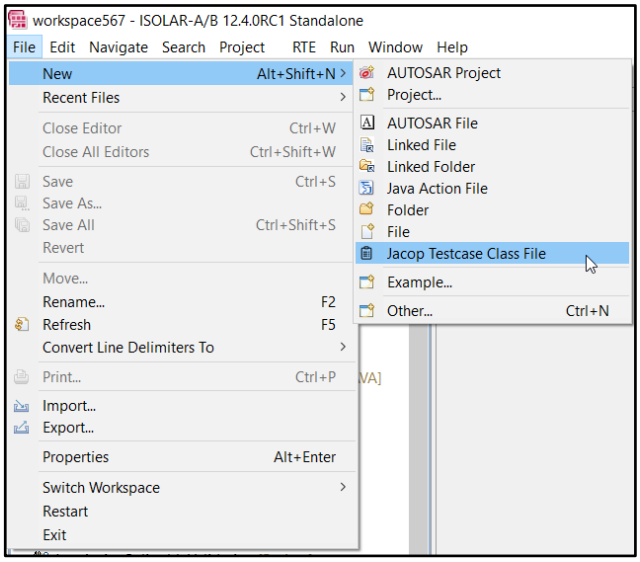

Figure 30 Context menu option to create a JaCoP testcase file

Select a folder and file name in the dialog, click finish to create file.

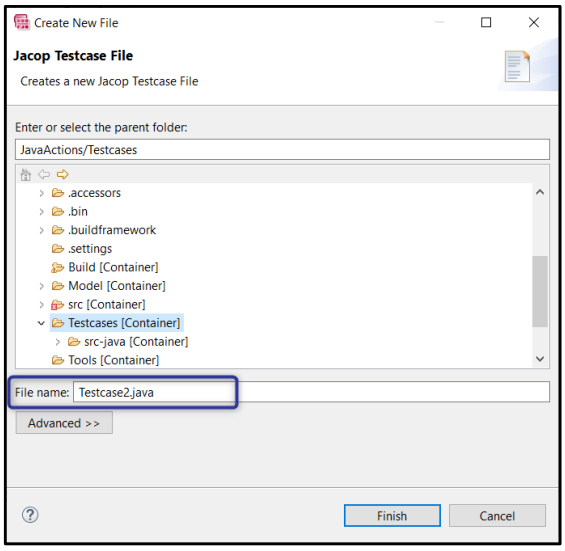

Figure 31 Create New File Wizard

Step-3: Testcase file will be created in the selected folder, with default methods. Modify testcases or add new test methods.

```
//Testcase2.java
public class Testcase2 extends JacopTestBase {

    @Override
    public List<String> getListOfExternalInputFilePaths() {
        return null;
    }

    @Override
    public List<String> getListOfExternalRefOuts() {
        return null;
    }

    @Override
    public List<String> getListOfInternalInputFilePaths() {
        List<String> paths = new ArrayList<>();
```

```
        paths.add("\\InputFiles"); // Ex., relative path to the project,
make sure the folder exists
        return paths;
    }

    @Override
    public List<String> getListOfInternalRefOuts() {
        List<String> paths = new ArrayList<>();
        paths.add("\\ReferenceFiles"); // Ex., relative path to the project,
make sure the folder exists
        return paths;
    }

    @Test
    public void testJacopAction() throws Exception{
        log("Running testJacopAction.");
        //  Your testcase implementation below
    }
}
```

Test Method Usage

Testcase should be written with provided API by the framework. For more details about test methods refer section 6.4

6.3 Modifying Created Test Class

In the newly created test class, 4 methods would be visible. These methods are necessary for copying the input and output files into project location. If these methods are not to be used, simply returning null is sufficient. A description of each method is described below

getListOfExternalInputFilePaths(): A list of string which contains paths of input files like (Configuration files, paramdef files).arxml,.xml and also inputs for Generate action execution like (.ftl, .txt) present outside the PVER have to be created if available and this list can be returned.

getListOfExternalRefOuts(): A list of string which contains paths to reference files present outside the PVER must be created if available, and this list can be returned. This list is required to support the comparison of reference files with output files.

getListOfInternalInputFilePaths(): A list of strings that contains paths to input files like (Configuration files, paramdef files).arxml,.xml and also inputs for Generate action execution like (.ftl, .txt) present within the PVER have to be created if available, and this list can be returned.

getListOfInternalRefOuts(): A list of string that contains paths to reference files present within the PVER has to be created if available, and this list can be returned. This list is required to support the comparison of reference files with output files

// SampleForMandatoryMethods class code snippet

```
public class SampleForMandatoryMethods extends JacopTestBase {

    @Override
    public List<String> getListOfExternalInputFilePaths() {
        List<String> listOfExternalInputFilePaths = new ArrayList<>();
        listOfExternalInputFilePaths.add("c:\\temp\\resources\\external");
        listOfExternalInputFilePaths.add("c:\\temp\\resources\\exteranl2");
        return listOfExternalInputFilePaths;
    }

    @Override
    public List<String> getListOfExternalRefOuts() {
        List<String> listOfExteranlRefOuts = new ArrayList<>();
```

```
        listOfExteranlRefOuts.add("c:\\temp\\resources\\refFiles");
        return listOfExternalRefOuts;
    }

    @Override
    public List<String> getListOfInternalInputFilePaths() {
        List<String> listOfInternalInputFilePaths = new ArrayList<>();
        listOfInternalInputFilePaths.add("\\inputFiles");
        return listOfInternalInputFilePaths;
    }

    @Override
    public List<String> getListOfInternalRefOuts() {
        List<String> listOfInternalRefOuts = new ArrayList<>();
        listOfInternalRefOuts.add("\\refFiles");
        return listOfInternalRefOuts;
    }

    @Test
    public void testJacopAction() throws Exception{
        log("Running testJacopAction.");
        //  Your testcase implementation below
    }
}
```

A test method should have an action which will perform the java action. Checking on different conditions performed during java action execution can be achieved with utils shown below:

- **JacopTestFactory**
- **TestGeneratorUtil**
- **TestForwarderUtil**
- **TestParamUtil**
- **TestFrameworkUtil**

6.4 How to Use Util Classes

A general structure of the testcase has been shown below. The test method would have an action to be performed and some conditions that have to be checked to make sure the action has run properly.

General Structure :

```
public class SampleTestClass extends JacopTestBase {
    @Override
    public List<String> getListOfExternalInputFilePaths() {
        List<String> listOfExternalInputFilePaths = new ArrayList<>();
        listOfExternalInputFilePaths.add(\\resources\\project\\external);
        listOfExternalInputFilePaths.add(\\resources\\project\\exteranl2);
        return listOfExternalInputFilePaths;
    }

    @Override
    public List<String> getListOfExternalRefOuts() {
        List<String> listOfExteranlRefOuts = new ArrayList<>();
        listOfExteranlRefOuts.add(\\externalOut\\refFiles);
        return listOfExternalRefOuts;
    }

    @Override
    public List<String> getListOfInternalInputFilePaths() {
        List<String> listOfInternalInputFilePaths = new ArrayList<>();
        listOfInternalInputFilePaths.add("\\inputFiles");
        return listOfInternalInputFilePaths;
    }

    @Override
```

```
    public List<String> getListOfInternalRefOuts() {
        List<String> listOfInternalRefOuts = new ArrayList<>();
        listOfInternalRefOuts.add("\\refFiles");
        return listOfInternalRefOuts;
    }

    public void testMyMethod() throws Exception{
        //Creating an action
        JavaExampleEcucValueAccess validate = new
JavaExampleEcucValueAccess();
        // running the created action
        validate.run(getContext());
        // Checking for some condition passed, here checking for log
generated
        IFile logFileCheck = JacopTestFactory.getCustomLogger();
        assertTrue(TestGeneratorUtil.contentContains(logFile1, "Starting
JavaExampleEcucValueAccess.java"));
    }
}
```

In the above test method different util classes like JacopTestFactory and TestGeneratorUtil are used. These utils have been used to achieve the purpose of checking if the validate action used in the above example is running properly. To know about different utilities available in util classes, check below.

6.4.1 JacopTestFactory

JacopTestFactory provides different API's that can be used for fetching files. There are different APIs provided that provide provisions in fetching log, reference or output files. There are APIs for fetching currently used project as well. More information about the methods available are shown below

getProject() : Gives the current working project ReturnType: IProject

getOutFile(String fileName) : Fetches the provided out file from the project, it will check the _out folder and fetches the file if present ReturnType: IFile

getLogFile(String fileName) : Fetches the provided log file from the project, it will check the _log folder and fetches the file if present ReturnType: IFIle

getRefFile(String fileName) : Fetches the provided ref file from the project, it will check the _ref folder and fetches the file if present ReturnType: IFile

getCustomLogger() : Fetches the log file which contains the information logged in the console during testcase execution. It will check for _log/TestLogger.txt file ReturnType: IFile

JacopTestFactoryExample :

```
public void testTestJacopTestFactoryApi() throws Exception {.
    // Fetching the project using Util
    IProject project = JacopTestFactory.getProject();
    // Checking the same project name
    assertEquals(project.getName(), getProjectName());

    // Creating and running an action
    JavaExampleArtopAccess artop = new JavaExampleArtopAccess();
    artop.run(getContext());

    // Checking for files generated
    assertNotNull(JacopTestFactory.getOutFile("test.txt");
    assertNotNull(JacopTestFactory.getLogFile("test.log");
    assertNotNull(JacopTestFactory.getRefFile("test.txt");
    assertNotNull(JacopTestFactory.getCustomLogger());
}
```

6.4.2 TestGeneratorUtil

TestGeneratorUtil provides different utilities that help in comparing the generated files. There are methods under this Util classes which can fetch contents of the file or check contents of the file. Below is description of available APIs,

contentEquals(IFile file1, IFile file2) : Checks if the contents in file1 and file2 are same. Returns "true" if same, "false" otherwise Return Type: boolean

contentContains(IFile file, String contentToCheck) : Checks if the content to check is available in the provided file Return Type: boolean

getContents(IFile file) : Provides all the contents available in the file as string Return Type: String

TestGeneratorUtilExample :

```
public void testTestGeneratorUtilApi() throws Exception {

    // Creating and running an action
    JavaExampleArtopAccess artop = new JavaExampleArtopAccess();
    artop.run(getContext());

    // Fetching different files from different locations
    IFile file1 = this.testProject.getFile("Build\\OutputFile.txt");
    IFile file2 = this.testProject.getFile("Build\\OutputFile.txt");
    IFile file3 = this.testProject.getFile("InputFile.txt");

    // Checking the results for different util methods
    assertTrue(TestGeneratorUtil.contentEquals(file1, file2);
    assertFalse(TestGeneratorUtil.contentEquals(file1, file3));
    assertTrue(TestGeneratorUtil.contentContains(file3, "Content in file
3"));
    assertFalse(TestGeneratorUtil.contentContains(file3, "Not available in
file 3"));
    assertFalse(TestGeneratorUtil.getContents(file3).isEmpty());
}
```

6.4.3 TestForwarderUtil

TestForwarderUtil class provides various utilities to fetch the modules, containers, parameters or references available in the forwarded file. Methods which can be used are provided below

getCreatedModules() : Fetches all the created modules by checking the forwarded resources Return Type: List<GModuleConfiguration>

getCreatedModule(String moduleShortName) : Fetches the module with provided short name Return Type: GModuleConfiguration

getCreatedContainer() : Fetches all the containers created by checking the forwarded resources Return Type: List<GContainer>

getCreatedContainer(String containerShortName) : Fetches the container matching the provided short name Return Type: GContainer

getCreatedParameters() : Fetches all the parameters by checking the forwarded resources Return Type: List<GParameterValue>

getCreatedParameters(String parameterShortName) : Fetches the parameter matching the provided short name Return Type: GParameterValue

getCreatedReference() : Fetches all the references by checking the forwarded resources Return Type: List<GConfigReferenceValue>

getCreatedReference(String referenceShortName) : Fetches the reference matching in provided short name Return Type: GConfigReferenceValue

TestForwarderUtil :

```
public void testTestForwarderUtilApi() throws Exception {

    // Creating and running an action
    JavaExampleArtopAccess artop = new JavaExampleArtopAccess();
    artop.run(getContext());

    // Fetching all the modules, containers, parameters, and references
    List<GModuleConfiguration> createdModules =
TestForwarderUtil.getCreatedModules();
    List<GContainer> createdContainer =
TestForwarderUtil.getCreatedContainer();
    List<GParameterValue> createdParameters =
TestForwarderUtil.getCreatedParameters();
    List<GConfigReferenceValue> createdReference =
TestForwarderUtil.getCreatedReference();

    // Checking for the size of list
    assertEquals(createdModules.size(), 3);
    assertEquals(createdContainer.size(), 4);
    assertEquals(createdParameters.size(), 2);
    assertEquals(createdReference.size(), 2);

    // Fetching the specific module, container, parameter, or reference
    assertNotNull(TestForwarderUtil.getCreatedModule("moduleShortName"));

assertNotNull(TestForwarderUtil.getCreatedContainer("containerShortName"));

assertNotNull(TestForwarderUtil.getCreatedParameters("parameterShortName"));

assertNotNull(TestForwarderUtil.getCreatedReference("referenceShortName"));
}
```

6.4.4 TestParamUtil

TestParamUtil class provides different utilities for mocking different types of parameter or references. This util helps in writing a mockito testcases easily and efficiently. To write a mockito testcase, an action and a context has to be created. For fetching a context, the JacopTestBase provides an api called *getContext()*. Other methods available under this util classes can be found below

mockStringParam(String stringValue) : Mocks the string parameter of a container. The value to be returned on mocking a string parameter is passed as the argument stringValue Return Type: IStringParameter

mockIntegerParam(int integerValue) : Mocks the integer parameter of a container. The value to be returned on mocking an integer parameter is passed as the argument integer-Value Return Type: IIntegerParameter

mockFloatParam(double doubleValue) : Mocks the float parameter of a container. The value to be returned on mocking a float parameter is passed as the argument dou-bleValue Return Type: IFloatParameter

mockEnumParam(String stringValue) : Mocks the enum parameter of a container. The value to be returned on mocking an enum parameter is passed as the argument stringValue Return Type: IEnumParameter

mockBooleanFalse() : Returns false while mocking a boolen parameter of a container Re-turn Type: IBooleanParameter

mockBooleanTrue() : Returns true while mocking a boolen parameter of a container Return Type: IBooleanParameter

6.4.4.1 Writing a testcase by mocking

With Mockito, a certain method can be mocked and mentioned what it has to return when that method is called. Using such methodology, we can mock container creation and replicate a behavior of a container without it being present. Same behavior can be replicated to short name and parameter or reference values. With Mockito we will mock an action and define what value should be returned when a method is triggered as part of mocking of an action. For better understanding on mocking, refer the code snippets below:

MockingShortNameExample :

```
public void testMockingShortName() throws Exception {

    // Creating a mock action
    JavaExampleEcucModelTestMocks mockAction = new
JavaExampleEcucModelTestMocks();

    // Getting the context and model
    IContext context = getContext();
    EcucValues model = context.getModel(EcucValues.class);

    mockAction.run(context);
    IFile file = this.testProject.getFile("Build\\OutputFile.txt");
    assertTrue(TestGeneratorUtil.contentContains(file, "shortName"));

    // Mocking the short name of moduleA so as to return newName
    Mockito.doReturn("newName").when(model.getModuleA()).getShortName();
    mockAction.run(context);
    assertFalse(TestGeneratorUtil.contentContains(file, "shortName"));
    assertTrue(TestGeneratorUtil.contentContains(file, "newName"));

}
```

MockingParameterExample:

```
public void testMockingParameterValue() throws Exception {

    // Getting the context and model
    IContext context = getContext();
    EcucValues model = context.getModel(EcucValues.class);

    // Creating a parameter and container
    Container1 container =
model.getModuleA().createContainer1("shortNameNew");
    container.createIntegerItem(5);
    // Checking integer parameter is created
    assertEquals(container.getIntegerItem().getValue().getIntValue(), 5);

    // Mocking the parameter to return 10

Mockito.doReturn(TestParamUtil.mockIntegerParam(10)).when(container).getInte
gerItem();
    // Fetching the container again to make sure mocked value will be
reflected
    container = getContainer1(model, "shortNameNew");
    assertEquals(container.getIntegerItem().getValue().intValue(), 10);
}

public container1 getContainer(EcucValues model, String shortName) {
    List<Container1> container1s = model.getModuleA().getContainer1s();
    Container1 container = null;
    for (Container1 container1 : container1s) {
      if (container1.getShortName().contains(shortName)) {
        container = container1;
```

```
        break;
      }
    }
    return container;
}
```

In the above example value to container has been assigned twice. This is because after creating an integer parameter we are trying to mock the created parameter for a container, which was fetched before creating its parameter.

MockingReferenceExample

```
public void testMockingReference() throws Exception {
    IContext context = getContext();
    EcucValues model = context.getModel(EcucValues.class);
    Container1 container = getContainer1(model, "shortName");
    Container3 container3 = model.getModuleA().getContainer3();
    Container2 container2 = model.getModuleA().getContainer2();
    // container.getSingleRef refers to container2
    assertEquals(container.getSingleRef().toString(),
container2.toString());

    IReferenceParameter singleRef = container.getSingleRef();
    Mockito.doReturn(container3).when(singleRef).followReference();

    assertEquals(container.getSingleRef().followReference().getShortName(),
container3.getShortName());
}

public container1 getContainer(EcucValues model, String shortName) {
    List<Container1> container1s = model.getModuleA().getContainer1s();
    Container1 container = null;
    for (Container1 container1 : container1s) {
      if (container1.getShortName().contains(shortName)) {
        container = container1;
        break;
      }
    }
    return container;
}
```

MockingValuesWithTestParamUtilExample :

```
public void testMockingValues() throws Exception {

   // Creating a mock action
   JavaExampleEcucModelTestMocks mockAction = new
JavaExampleEcucModelTestMocks();

   // Getting the context
   IContext context = getContext();
   EcucValues model = context.getModel(EcucValues.class);

   mockAction.run(context);
   List<String> infos = context.getLog().getInfos();

   // Checking the values of contents before mocking
   assertTrue(infos.contains("model.getModuleA().getShortName()=ModuleA"));
   assertTrue(infos.contains("container.getShortName()=shortName"));
   assertTrue(infos.contains("container.getStringItem()=stringItem"));
   assertTrue(infos.contains("container.getIntegerItem()=3"));
   assertTrue(infos.contains("container.getBooleanItem()=true"));
   // Here Float parameter is absent for containerB
   assertFalse(infos.contains("container.getFloatItem()=1.111"));

   // Changing the values of content with mocking

Mockito.doReturn("newMockedShortName").when(model.getModuleA()).getShortName
();
```

```
Mockito.doReturn("newMockedContainerName").when(container).getShortName();

Mockito.doReturn(TestParamUtil.mockStringParam("newMockedStringItem")).when(
container).getStringItem();

Mockito.doReturn(TestParamUtil.mockIntegerParam(4)).when(container).getInteg
erItem();

Mockito.doReturn(TestParamUtil.mockFloatParam(1.111)).when(container).getFlo
atItem();

Mockito.doReturn(TestParamUtil.mockBooleanFalse()).when(container).getBoolea
nItem();

   // Clearing the previous values from infos
   infos.clear();
   mockAction.run(context);

   // Checking if the values changed

assertTrue(infos.contains("model.getModuleA().getShortName()=newMockedShortN
ame"));

assertTrue(infos.contains("container.getShortName()=newMockedContainerName")
);

assertTrue(infos.contains("container.getStringItem()=newMockedStringItem"));
   assertTrue(infos.contains("container.getIntegerItem()=4"));
   assertTure(infos.contains("container.getFloatItem()=1.111"));
   assertTrue(infos.contains("container.getBooleanItem()=false"));
 }
```

ClearMocksExample

```
// Creating a mock action
JavaExampleEcucModelTestMocks mockAction = new
JavaExampleEcucModelTestMocks();

// Getting the context, model and container
IContext context = getContext();
EcucValues model = context.getModel(EcucValues.class);
mockAction.run(context);
List<String> infos = context.getLog().getInfos();

// Checking the values of contents before mocking
assertTrue(infos.contains("model.getModuleA().getShortName()=ModuleA"));
// Changing the values of content with mocking
Mockito.doReturn("newMockedShortName").when(model.getModuleA()).getShortName
();

// Clearing the previous values from infos
infos.clear();
mockAction.run(context);
// Checking the values of contents after mocking
assertTrue(infos.contains("model.getModuleA().getShortName()=newMockedShortN
ame"));

// Clearing the mocks
TestFrameworkUtil.clearMocks()
// Clearing the previous values from infos
infos.clear();
mockAction.run(context);
// Checking the values for contents after clearing the mock
assertTrue(infos.contains("model.getModuleA().getShortName()=ModuleA"));
```

6.4.5 TestFrameworkUtil

TestFrameworkUtil is a general util class which can be used for copying and searching of files or folders. Below are the methods available:

- exchangeInputFile(IContext context, String file1Path, String file2Path) : Exchanges the file1 with file2, so file2 can be accessed through file1. Once this method is called file1 will always point to file2.
- clearExchangeInputFiles(IContext context) : Clears any exchange that was performed earlier
- clearMocks() : Clears all the Mocking performed on objects
- getAllParamdefsInProject() : Returns a list of all parameter definitions available in the project as a list of string. This list will be having the URL's of parameter definition. Return Type: List<String>

TestFrameworkUtilExample

```
public void testTestFrameworkUtilApi() throws Exception {

    // Creating and running an action
    JavaExampleArtopAccess artop = new JavaExampleArtopAccess();
    artop.run(getContext());

    // Getting the context
    IContext context = getContext();

    TestFrameworkUtil.exchangeInputFile(context, "hello_txt.ftl",
"/inputFiles/InputFile.txt");
    // Here the file fetched will be InputFile.txt
    inputFile = context.getInputFile("hello_txt.ftl");
    assertTrue(inputFile.getContent().contains("This is a textfile that can
be read from a JavaAction"));

    // After clearing hello_txt.ftl will be fetched rather than
InputFile.txt
    TestFrameworkUtil.clearExchangeInputFiles(context);
    inputFile = context.getInputFile("hello_txt.ftl");
    assertFalse(inputFile.getContent().contains("This is a textfile that can
be read from a JavaAction"));
}
```

getAllParamdefsInProject() should be used inside the overridden method getListOfInternalInputFilePaths. So, if all the parameter definitions need to be considered then this api can be returned. In case if only certain parameter definitions need to be returned, a filter logic can be implemented in getListOfInternalInputFilePaths and return the filtered value. An example for using getAllParamdefsInProject() has been illustrated below:

getAllParamdefsInProjectExample

```
public class SampleTestClass extends JacopTestBase {
    @Override
    public List<String> getListOfExternalInputFilePaths() {
        List<String> listOfExternalInputFilePaths = new ArrayList<>();
        listOfExternalInputFilePaths.add(\\resources\\project\\external);
listOfExternalInputFilePaths.add(\\resources\\project\\exteranl2);
        return listOfExternalInputFilePaths;
    }

    @Override
    public List<String> getListOfExternalRefOuts() {
        List<String> listOfExteranlRefOuts = new ArrayList<>();
        listOfExteranlRefOuts.add(\\externalOut\\refFiles);
        return listOfExternalRefOuts;
    }

    @Override
    public List<String> getListOfInternalInputFilePaths() {
```

```
        List<String> listOfInternalInputFilePaths = new ArrayList<>();
        listOfInternalInputFilePaths.add("\\inputFiles");
        // Adding all the paramdefs in project, hence every paramdef's will
be considered

listofInternalInputFilePath.addAll(TestFrameworkUtil.getAllParamdefsInProjec
t());
        return listOfInternalInputFilePaths;
    }

    @Override
    public List<String> getListOfInternalRefOuts() {
        List<String> listOfInternalRefOuts = new ArrayList<>();
        listOfInternalRefOuts.add("\\refFiles");
        return listOfInternalRefOuts;
    }

    public void testMyMethod() throws Exception{
        //Creating an action
        JavaExampleEcucValueAccess validate = new
JavaExampleEcucValueAccess();
        // running the created action
        validate.run(getContext());
        // Checking for some condition passed, here checking for log
generated
        IFile logFileCheck = JacopTestFactory.getCustomLogger();
        assertTrue(TestGeneratorUtil.contentContains(logFile1, "Starting
JavaExampleEcucValueAccess.java"));
    }
}
```

getAllParamdefsInProjectFilterLogicExample

```
@Override
   public List<String> getListOfInternalInputFilePaths() {
       List<String> listOfInternalInputFilePaths = new ArrayList<>();
       listOfInternalInputFilePaths.add("\\inputFiles");
       // Filtering out all parameter definition which have "com" in its URI
       List<String> allParamdefsInProject =
TestFrameworkUtil.getAllParamdefsInProject();
       for(String paramdefsInProject : allParamdefsInProject) {
           if(paramdefsInProject.contains("Com")) {
               listOfInternalInputFilePaths.add(paramdefsInProject);
           }
       }
       return listOfInternalInputFilePaths;
   }
```

6.5 Execution of testcases

6.5.1 How to run the testcase in GUI ?

- Right click on the test class and Run As > JaCoP Test

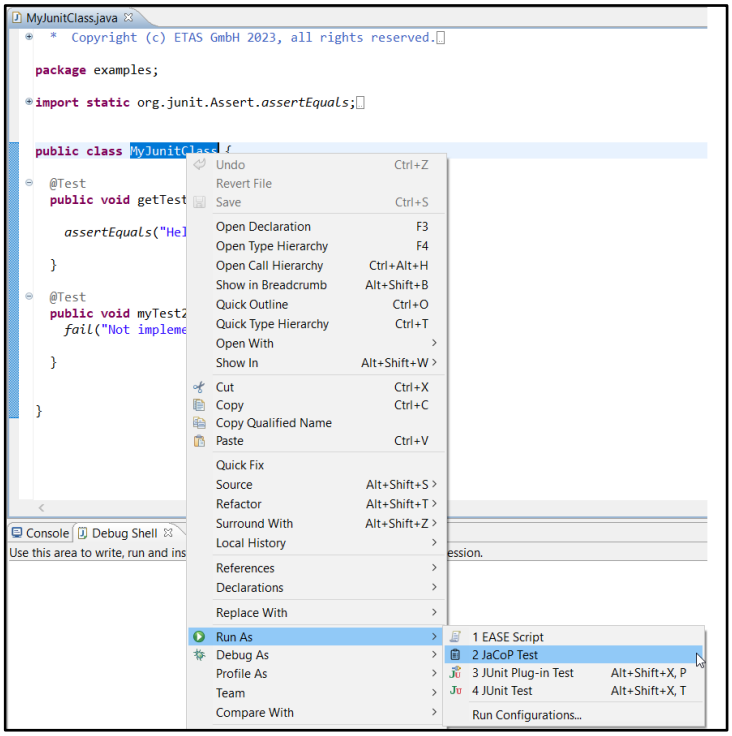

Figure 32 Option to run JaCoP Test

To rerun a testcases : **Run As > JaCoP Test** option creates a debug configuration. Testcase can be run using debug configuration launcher menu as shown in Figure

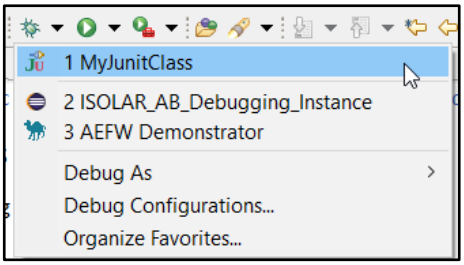

Figure 33 Option to rerun testcase in debug launcher menu

Results of the testcase run can be viewed under the Junit tab as shown in Figure 35

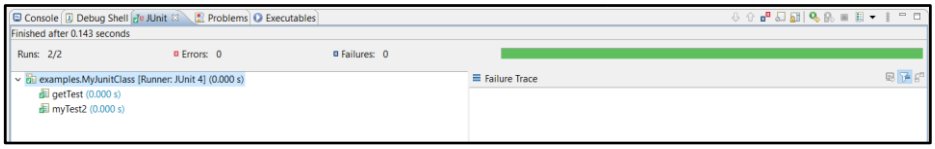

Figure 34 Junit tab showing the testcase results

6.5.2 How to run the testcase in CLI mode

Execute in CLI using the following command

`ISOLAR-B.cmd -jacoptest -TESTPROJECT=testPluginProjectPath -TESTREPORTFOLDER=reportFolderPath`

optional: -LAUNCHFILENAME=launchFileName

optional: -Coverage=coverageFormat

Below shown is the help document when the ISOLAR-B.cmd -jacoptest -h command is run in CLI

```
C:\Users\Downloads\ISOLAR-AB-win32.win32.x86_64>echo off
CLI command: -jacoptest
Tool name: ISOLAR-B

Short Desc: Testcase execution and report generation.

Description: JunitTestcase execution and report generation command

Syntax: ISOLAR-B -jacoptest [Parameters...]

  --testProject      :  [Mandatory]  Path of the root folder which contains the
test plugin project.
  --testReportFolder :  [Mandatory] Path of the local workspace location where
the test reports needs to be generated
  --launchFileName   :  [Optional]  Name of the launch file under the test plugin
project.
  --coverage         :  [Optional] Records coverage information and generates
them in given format. The default format is html. The supported formats are CSV,
HTML, HTMLZIP and XML. eg. --coverage=html

Return Values:
  0 :  Command executed sucessfully.
  1 :  Error while executing the command.

Example:

            ISOLAR-B -jacoptest --testPro-
ject=C:\temp\Jacop_Test\com.bosch.java.test.framework.sample.test --testReport-
Folder=C:\temp\Report



The system cannot find the batch label specified - exit
```

Example:

```
ISOLAR-B.cmd -jacoptest -TESTPROJECT=C:\TestData\TestProjectForExecution -
TESTREPORTFOLDER= C:\temp\Report
```

The below info would be displayed on successful execution of the -jacoptest command.

Results :

```
[INFO]: Report is exported successfully to the location : C:\temp\Report\TestRe-
sults.xml
[INFO]: Console log is available at C:\temp\Report\console.log
```

7 Contact Information

Technical Support

For details of your local sales office as well as your local technical support team and product hotlines, take a look at the website: www.etas.com/hotlines

ETAS Headquarters

ETAS GmbH
Borsigstraße 24
70469 Stuttgart
Germany

Phone: +49 711 3423-0
Fax: +49 711 3423-2106
Internet: www.etas.com

Figures

Tables

Index

No index entries found.

Revision History